# Informe de Laboratorio: Estructura de Computadores

**Nombre del Estudiante:** [Eivar Solis Tulande]
**Fecha:** [26/02/2026]
**Asignatura:** Estructura de Computadores

**Enlace del repositorio en GitHub:** [https://github.com/Eivars10jf/eivar-solis-estructura-computadores-act01]

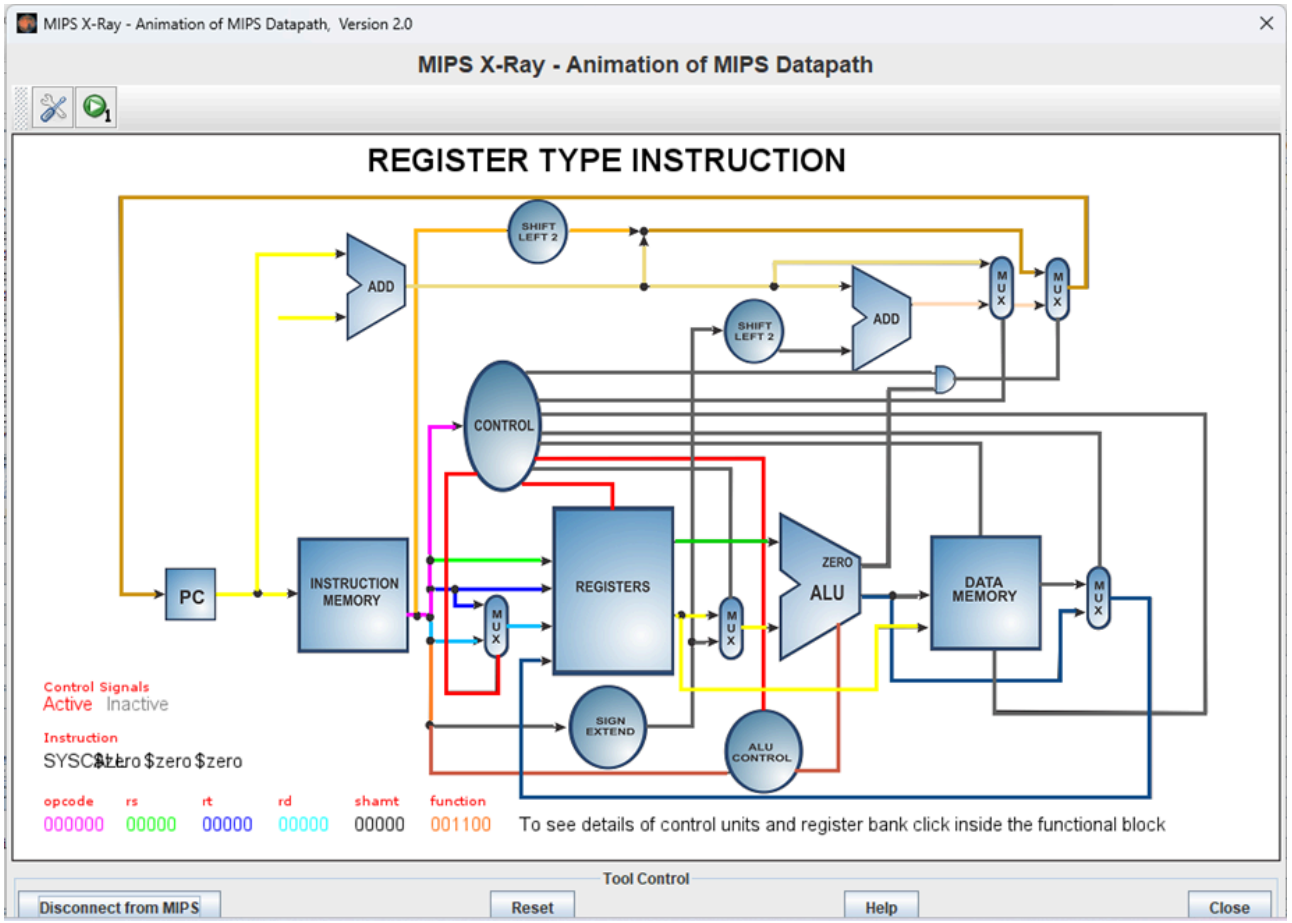
---

## 1. Análisis del Código Base

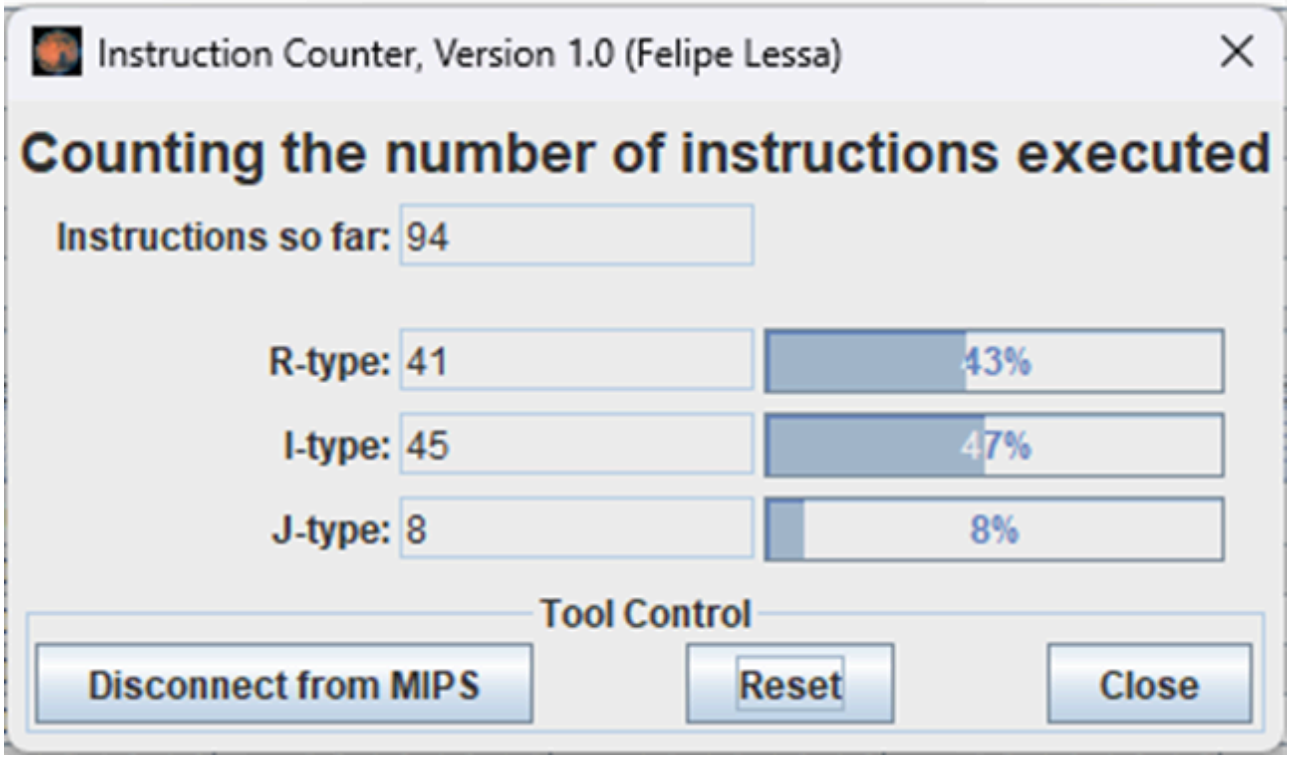
### 1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantalla de la ejecución del `programa_base.asm` utilizando las siguientes herramientas de MARS:

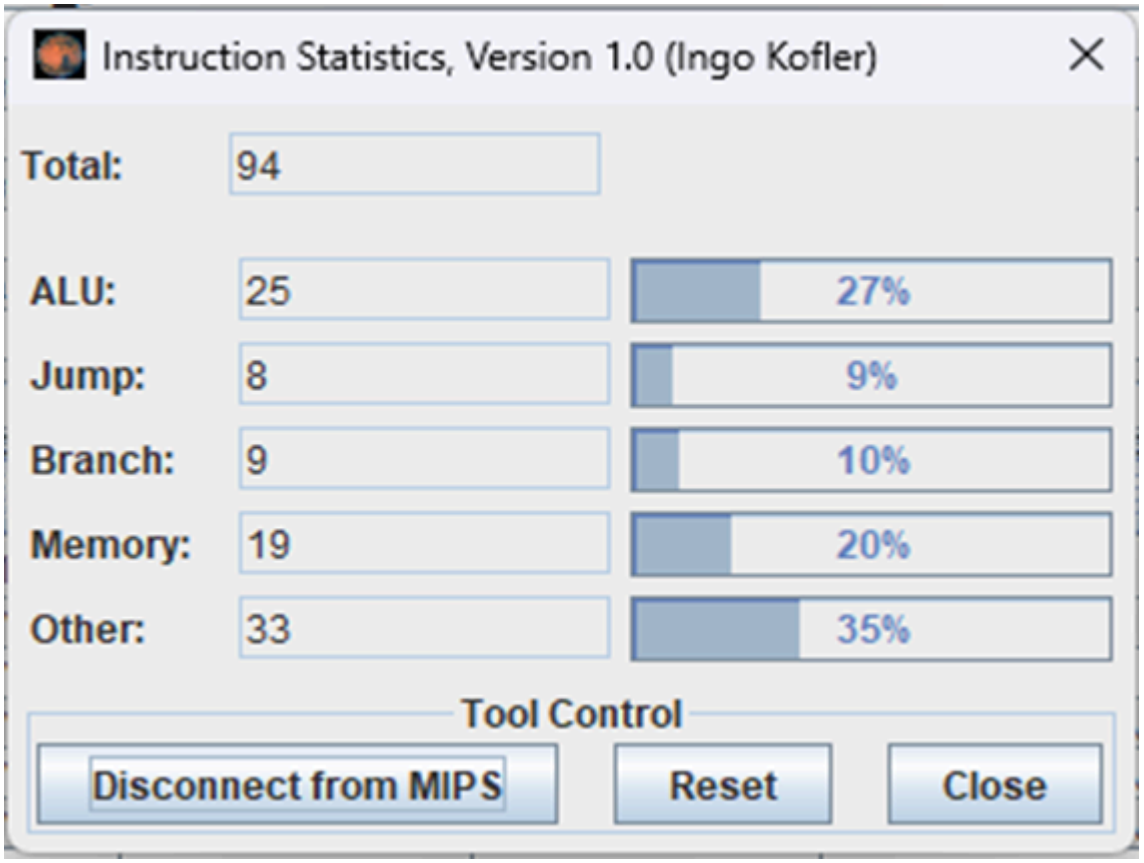
- **MIPS X-Ray** (Ventana con el Datapath animado).



- **Instruction Counter** (Contador de instrucciones totales).




- **Instruction Statistics** (Desglose por tipo de instrucción).




## 1.2. Identificación de Riesgos (Hazards)

Completa la siguiente tabla identificando las instrucciones que causan paradas en el pipeline:

| Instrucción Causante | Instrucción Afectada | Tipo de Riesgo (Load-Use, etc.) | Ciclos de Parada |
|---|---|---|---|
| `lw $t6, 0($t5)` | `mul $t7, $t6, $t0` | Load-Use | (1) |

| Instrucción Causante | Instrucción Afectada | Tipo de Riesgo (Load-Use, etc.) | Ciclos de Parada |
|---|---|---|---|
| `mul $t7, ($t6) $t0` | `addu $t8, $t7, $t1` | Dependencia de datos (RAW) | (0) Resuelto Optm. |

### 1.2. Estadísticas y Análisis Teórico

Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) varía. Completa la siguiente tabla de análisis teórico:

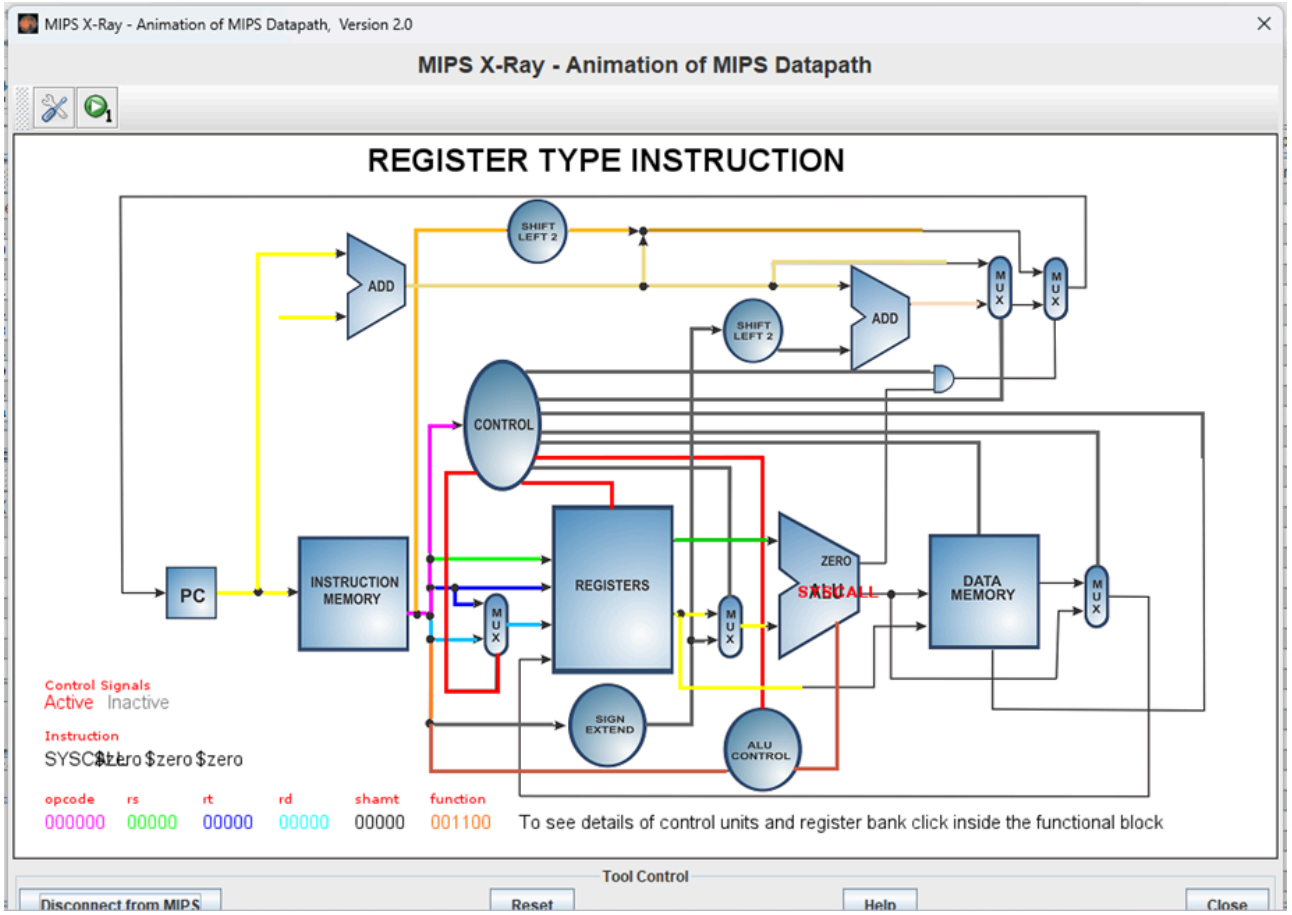
| Métrica | Código Base | Código Optimizado |
|---|---|---|
| Instrucciones Totales (según MARS) | 94 | 94 |
| Stalls (Paradas) por iteración | 1 | 0 |
| Total de Stalls (8 iteraciones) | 8 | 0 |
| **Ciclos Totales Estimados** (Inst + Stalls) | 102 | 94 |
| **CPI Estimado** (Ciclos / Inst) | ~1.08 | 1.00 |

## 2. Optimización Propuesta

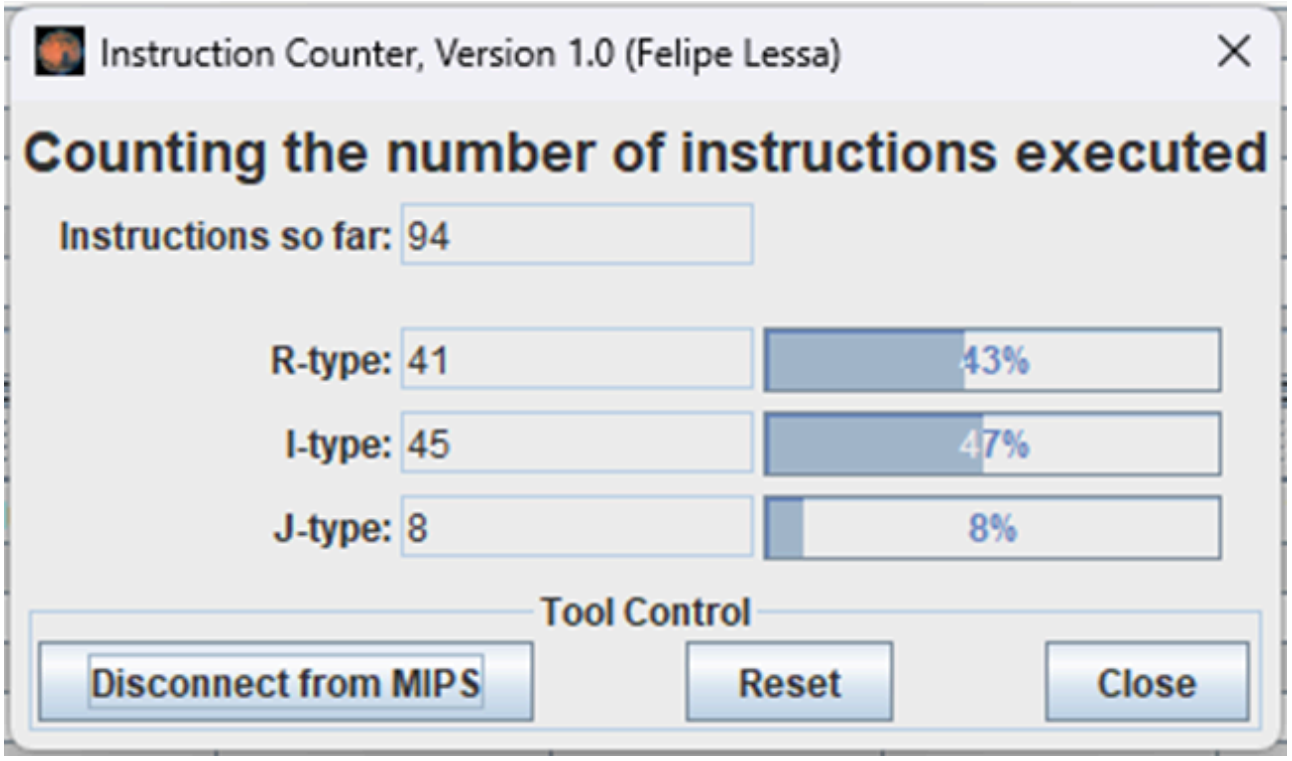
### 2.1. Evidencia de Ejecución (Código Optimizado)

Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.asm` utilizando las mismas herramientas que en el punto 1.1:

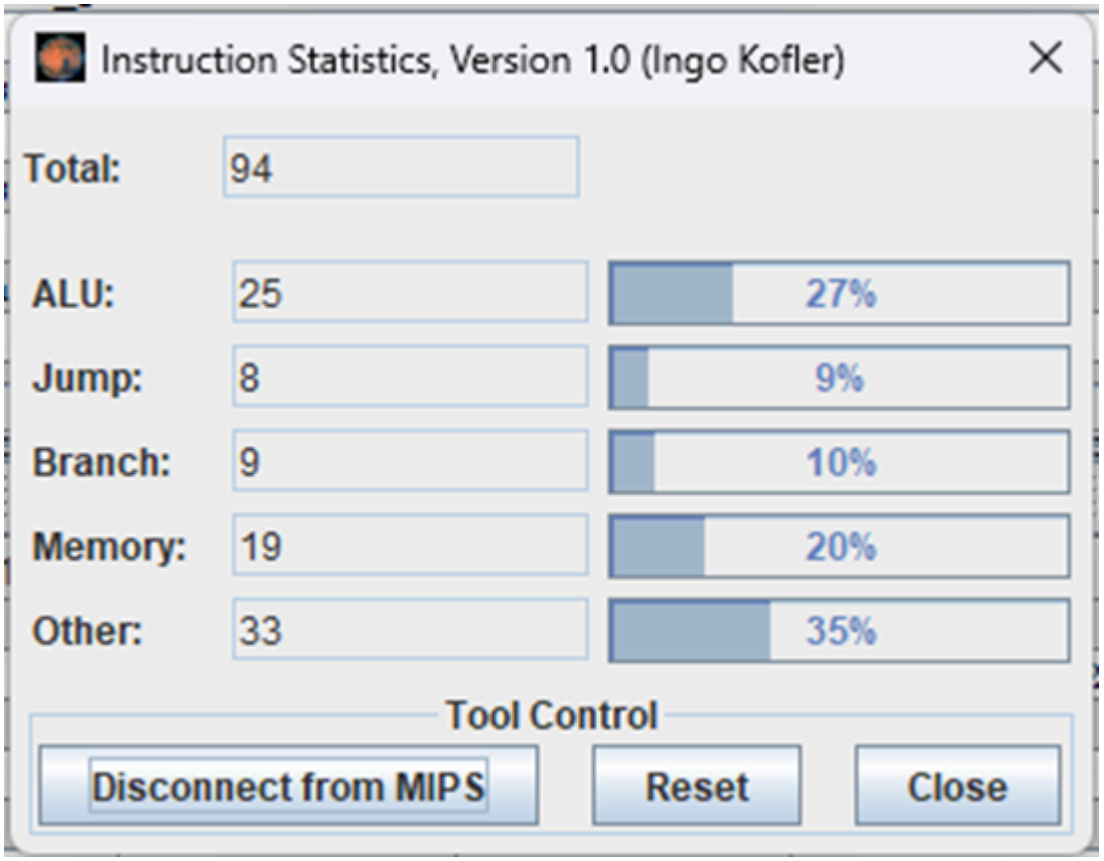
- **MIPS X-Ray**.




- **Instruction Counter**.



- **Instruction Statistics**.




## 2.2. Código Optimizado

Pega aquí el fragmento de tu bucle `loop` reordenado:

```
# Laboratorio: Estructura de Computadores - OPTIMIZADO
# Objetivo: Minimizar paradas (stalls) reordenando instrucciones.

.data
    vector_x: .word 1, 2, 3, 4, 5, 6, 7, 8
    vector_y: .space 32
    const_a:  .word 3
    const_b:  .word 5
    tamano:   .word 8

.text
.globl main

main:
    la $s0, vector_x
    la $s1, vector_y
    lw $t0, const_a
    lw $t1, const_b
    lw $t2, tamano
    li $t3, 0

loop:
    beq $t3, $t2, fin
```

```
    sll $t4, $t3, 2
    addu $t5, $s0, $t4

    # --- CARGA DE DATO ---
    lw $t6, 0($t5)

    # --- INSTRUCCIONES MOVIDAS PARA EVITAR STALL ---
    # Calculamos la dirección de destino de Y mientras esperamos que termine el lw
    addu $t9, $s1, $t4
    # Incrementamos el índice i anticipadamente
    addi $t3, $t3, 1

    # --- OPERACIÓN ARITMÉTICA (Ya no hay stall de Load-Use) ---
    mul $t7, $t6, $t0
    addu $t8, $t7, $t1

    # --- ALMACENAMIENTO ---
    sw $t8, 0($t9)

    j loop

fin:
    li $v0, 10
    syscall
```

### 2.2. Justificación Técnica de la Mejora

Explica qué instrucción moviste y por qué colocarla entre el `lw` y el `mul` elimina el riesgo de datos:

En el código base corrido en MARS 4.5, la instrucción mul $t7, $t6, $t0 intentaba leer el registro $t6 un ciclo después de que la instrucción lw $t6, 0 $t5 iniciara la carga desde memoria. En un pipeline de 5 etapas, el dato de un lw solo está disponible al final de la etapa MEM; por ello, al usarlo en la etapa EX del mul, inmediatamente después, el hardware se ve obligado a insertar un Stall.

Por lo anterior, se procedió a mover las instrucciones addu $t9, $s1, $t4 “cálculo de dirección de destino” y addi $t3, $t3, 1 “incremento del índice” para que se ejecuten después del lw. Al organizar estas instrucciones independientes en medio, se proporciona los ciclos necesarios para que el dato de $t6 avance en el pipeline. En tal sentido, cuando el mul llega a la etapa de ejecución, el dato ya puede ser obtenido mediante las rutas de Forwarding (adelantamiento), eliminando la necesidad de detener el procesador.

---

## 3. Comparativa de Resultados

| Métrica | Código Base | Código Optimizado | Mejora (%) |
|---|---|---|---|
| Ciclos Totales | 102 | 94 | 7,84% |
| Stalls (Paradas) | 8 | 0 | 100% |
| CPI | 1,08 | 100 | 7,41% |

---

# 4. Conclusiones

¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es siempre posible eliminar todas las paradas?

La segmentación (pipeline) obliga al programador a no solo pensar en la corrección del código, sino también en el orden temporal de las instrucciones. Un código lógicamente idéntico puede ser muy más lento si no se tiene en cuenta la microarquitectura del procesador. Como se demostró en este laboratorio, el reordenamiento de instrucciones permite aprovechar los recursos de la ALU mientras se espera por operaciones lentas de memoria, maximizando el rendimiento del mismo.

No obstante lo anterior, no siempre es posible eliminar todas las paradas, en algunos algoritmos con dependencias de datos muy densas, donde cada instrucción necesita obligatoriamente el resultado de la anterior o en saltos condicionales impredecibles “control hazards”, el procesador inevitablemente tendrá que insertar paradas o descartar instrucciones si el hardware de adelantamiento o el predictor de saltos no son suficientes para cubrir la latencia.

Es también importante indicar que, para efectos de desarrollo de este laboratorio, es que hay diferencias entre como funciona un procesador real y un simulador funcional “MARS 4.5”, el simulador valida que la lógica es correcta, se centra en contar cuantas instrucciones se ejecutan y no cuanto tardan estas, contrario a un procesador real. La diferencia entre ambos está en el CPI (Tiempo de CPU = IC x CPI x T), donde IC lo aporta el simulador en ambos códigos “base y optimizado”, En el CPI el código optimizado es mejor, ya que el código base tiene un CPI mayor a 1 (por paradas), mientras que el optimizado tiene un CPI de 1.

Pese a que las estadísticas en MARS 4.5 son idénticas para el “código base y optimizado”, ya que el simulador contabiliza la ejecución lógica del algoritmo; el código optimizado es superior en rendimiento, ya que reduce los stalls en el pipeline, logrando un menor número de ciclos totales y un CPI más eficiente.